Impl-1: Task Statement

AI-Assisted Implementation & Foundation Build for the Academic AI Compute Fabric

Course Activity · Group · 4–6 Hours · **BMad Method (Full Ceremony Build)**
Prerequisites: Completed Plan-2 and Solu-2
Deliverable: Working Codebase, Automated Test Suite, Scaffold Infrastructure, sprint-status.yaml, AI Decision Log, and Implementation Walkthrough Report.

---

1. Executive Summary & Implementation Framing

Moving from Specification to Verified Code

In **Plan-2**, your engineering team created the upstream specification spine: the Product Requirements Document (PRD), User Experience specifications (DESIGN.md, EXPERIENCE.md), Architectural Invariants (AD-n), OpenAPI 3.1 contracts, and prioritized Epics and Stories.

In **Impl-1**, your team crosses the threshold into **verifiable software implementation**. You will not merely generate scaffolding stubs or passive templates. Your objective is to build a **fully functional, running, and automatedly tested slice** of your assigned subsystem module.

> **Active Constraint Principle:** Upstream artifacts (PRD, OpenAPI contracts, PostgreSQL schemas, and Architectural Invariants) are **active execution constraints** for AI agents, not shelfware. Prompting an agent to *"build my application"* from scratch without continuous architectural grounding is the primary failure mode of AI-assisted engineering. In this task, you will govern your AI agents using the **BMad Build Full Ceremony** to produce resilient, high-craft code.

Scope of Impl-1: Foundation Scaffold + 3 Core Stories

To establish a solid engineering baseline, **Task Impl-1** requires each team to deliver: 1. **The Subsystem Foundation & Scaffolding:** A clean, containerized or locally runnable project structure adhering to Ports & Adapters (Hexagonal Architecture), containing database migrations, configuration management, simulated fleet fixtures, dependency validation, and health probes. 2. **Three Fully Developed Core Stories:** Implementation of the **three foundational user stories** recommended for your module (detailed in the Annex). These stories establish the operational core of the subsystem (happy paths, primary state transitions, data persistence, and essential validation gates) before complex failure recovery and advanced testing are tackled in Impl-2.

Standalone Engineering Mandate

Consistent with the pedagogical philosophy of the course: * **Zero Cross-Team Blocking:** Each subsystem operates completely independently. You do not wait for another team to deploy an API. * **Simulated Fleet & Hardware Agnosticism:** You do not need physical NVIDIA GPUs. All hardware metrics, driver states, and cluster telemetry are supplied via deterministic fixtures or simulated generators (31 workstations × 24 GB VRAM, 1 server × 2 × 48 GB VRAM). * **Deterministic Execution:** Any evaluator, instructor, or teammate must be able to clone your repository, run a single setup command, execute the automated test suite, and launch the service without manual environment tweaking.

---

2. Learning Objectives

By completing Impl-1, your team will demonstrate mastery of:

1. **Operationalizing Sprint Backlogs:** Translating upstream specifications into an active, machine-readable sprint-status.yaml tracking file using bmad-sprint-planning.
2. **Executing the BMad Build Full Ceremony:** Directing AI developer agents (**Amelia**, bmad-agent-dev) through the rigorous 6-step build lifecycle (Context Ingestion → Repository Inspection → Task Breakdown → Concurrent Code + Tests → DoD Verification → Review Report).
3. **Architectural Scaffolding & Boundary Isolation:** Structuring a clean codebase (FastAPI/Python or Node/TypeScript) with strict separation between Domain Logic, Inbound Adapters (REST/CLI), and Outbound Adapters (Database/Simulators).
4. **Contract-Bound Implementation:** Implementing data access layers, service controllers, and domain models that strictly conform to OpenAPI 3.1 contracts and SQL schemas without hallucinating extraneous endpoints or columns.
5. **Test-Driven Verification:** Writing and executing automated unit and integration tests (using Pytest or Vitest/Jest) that validate every Gherkin acceptance scenario (Given-When-Then) across the 3 stories.
6. **AI Governance & Code Walkthrough:** Auditing AI-generated diffs against architectural invariants, eliminating unvetted dependencies, rejecting sloppy boilerplate, and documenting code provenance in an AI Decision Log.

---

3. The BMad Build Full Ceremony Workflow

Implementation is governed by the **Full Ceremony Build** protocol (Unit 5 · Part 2). You must not issue unconstrained “code generation” prompts. Instead, every story must traverse the following structured cycle: plan, prepare (DoR), test design, create story, build solution, review, verify and gate.

The 6-Step Agent Execution Sequence inside bmad-build

For each of your 3 stories, you will supervise the development agent through these exact steps:

1. **Context Ingestion:** The agent loads the target story markdown, PRD requirements, OpenAPI contracts, and applicable Architectural Invariants (AD-n).
2. **Repository Inspection:** The agent inspects existing project conventions, folder structures, linters, schemas, and type definitions *before modifying files*.
3. **Granular Task Breakdown:** The agent formulates a sequence of incremental sub-tasks (e.g., Domain Model → Repository Interface → InMemory/SQL Adapter → API Endpoint → Test Suite).
4. **Concurrent Code + Tests:** The agent implements business logic, endpoints, and automated tests hand-in-hand. Production code is never written in isolation from test assertions.
5. **Definition of Done (DoD) Verification:** The agent executes the test runner (pytest or npm test) and linter (flake8/ruff or eslint). The task cannot be marked done until 100% of story acceptance criteria pass.
6. **Review Report & Status Promotion:** The agent updates sprint-status.yaml from in-progress to ready-for-review, providing a concise walkthrough of changes and diff evidence.

Ceremony Justification & The Two-Sentence Rule

In your submission, you will justify why each story followed the Full Ceremony.
> **The Two-Sentence Rule:** *"If you cannot clearly state the business goal, its blast radius/risk, and its automated verification in two simple sentences, it is NOT a compact change and MUST follow the Full Ceremony."*
Because the 3 Impl-1 stories establish shared domain models, database tables, and public API contracts, **Full Ceremony is mandatory**.

---

4. Technical Baseline & Scaffolding Standards

All student implementations must conform to the following technical foundation:

1. Project Organization (Clean Architecture / Ports & Adapters)

```
module-name/
├── README.md                    # Quick-start, setup commands, architecture summary
├── docker-compose.yml           # Standalone service + database runner
├── .env.example                 # Documented environment variables
├── pyproject.toml / package.json  # Approved dependency manifest
├── docs/
│   ├── prd.md                   # Scoped module PRD (from Plan-2)
│   ├── contracts/               # OpenAPI 3.1 YAML / JSON schemas
│   ├── sprint-status.yaml       # Active BMad sprint tracking file
│   └── ai-decision-log.md       # Documented AI prompts & rejected suggestions
├── src/
```

```
│   ├── domain/                 # Pure business logic, models, entities, state machines
│   ├── ports/                  # Abstract interfaces (Repositories, TelemetryProvider)
│   ├── adapters/
│   │   ├── inbound/            # FastAPI routers, CLI controllers, schemas/DTOs
│   │   └── outbound/           # SQLite/PostgreSQL repositories, Mock simulators
│   └── config.py               # Settings & environment validation
└── tests/
    ├── conftest.py             # Fixtures, test client, in-memory DB setups
    ├── fixtures/               # Static JSON seed data (nodes, users, policies)
    └── unit/                   # BDD-mapped automated tests for the 3 stories
```

2. Supported Technology Stacks

- **Python Track (Recommended):** Python 3.11+, FastAPI, Pydantic v2, SQLAlchemy 2.0 / SQLModel, Pytest, Alembic or SQLite in-memory for testing.
- **TypeScript Track:** Node.js 20+, Express or Fastify, Zod, Prisma or Drizzle ORM, Vitest or Jest.

3. Non-Negotiable Engineering Rules

- **Zero Hallucinated Endpoints:** Endpoint signatures, parameter names, and HTTP status codes must match your OpenAPI 3.1 contracts exactly.
- **Dependency Safety:** Do not add unvetted libraries. Ensure all imported packages are safe and common to modern web/microservice architectures.
- **Data Seed & Fixtures:** Your repository must include a fixtures/ directory containing the seed data representing the 32 university compute nodes (31 × 24 GB workstations, 1 × 2 × 48 GB server).
- **Clean Git Hygiene:** Commits must reflect incremental story progression (e.g., feat(scaffold): initialize ports and db schemas, feat(story-1.1): implement roster ingestion and tests, test(story-1.1): verify duplicate user conflict handling).

---

5. Deliverables & Acceptance Checklist

Required Deliverables

Each student team must submit a standalone GitHub repository (or organized archive) containing:

1. **Functional Source Code:** Complete, running codebase implementing the scaffold foundation and the 3 assigned stories.
2. **Automated Test Suite:** Pytest / Vitest test files covering all acceptance criteria across the 3 stories, with $\geq 90\%$ passing rate and zero skipped tests.
3. **sprint-status.yaml:** Updated sprint tracking file showing the progression of the 3 stories from ready-for-dev → in-progress → ready-for-review / done.
4. **Implementation Walkthrough Report (walkthrough-impl1.md):** A concise document summarizing:
   - Module architecture and structural directory layout.

- For each of the 3 stories: Goal, contract endpoints implemented, database changes, and test execution evidence.
- Output log or screenshot of the automated test runner passing.

5. **AI Decision Log (ai-decision-log.md):** Minimum 6 documented AI interactions, including at least **2 instances** where the team corrected an AI code hallucination, fixed an invalid assumption, or refactored an AI-generated implementation to satisfy an Architectural Invariant.

Student Self-Assessment Checklist (Definition of Ready for Submission)

Before submitting, verify that your project satisfies every check:

- **Execution:** docker-compose up or pytest runs out-of-the-box following only the instructions in README.md, no external files or configurations needed.
- **Scaffold Integrity:** Directory adheres strictly to Ports & Adapters separation; business logic does not directly import HTTP request objects or raw SQL drivers.
- **3 Core Stories Implemented:** All 3 recommended stories for your assigned module have working domain models, storage adapters, and API endpoints.
- **Contract Compliance:** API request/response payloads match the OpenAPI specification schema.
- **Test Coverage:** Every story has at least 2 distinct automated tests directly validating its Gherkin acceptance criteria.
- **Error Handling:** Non-happy paths (e.g., HTTP 400 Bad Request, 404 Not Found, 409 Conflict, 422 Unprocessable) return structured JSON error payloads, not unhandled stack traces.
- **BMad Traceability:** sprint-status.yaml is populated and reflects the real implementation status.
- **AI Governance:** AI Decision Log explicitly details prompts, outputs, and human review decisions.

6. Academic Evaluation Rubric (100 Points)

Assessment Dimension	Weight	Outstanding (90–100%)	Competent (75–89%)	Developing (60–74%)	Beginning (<60%)
1. Architecture & Scaffolding	**20%**	clear Ports & Adapters architecture. Clean dependency injection, modular settings, automated	Clear structure with minor leakage across layers (e.g., controller	Monolithic file organization; framework objects tightly coupled to business models. Hard to run locally	Non-runnable project; chaotic folder structure; missing dependenci

Assessment Dimension	Weight	Outstanding (90–100%)	Competent (75–89%)	Developing (60–74%)	Beginning (<60%)
		migrations/tables, and clear separation between domain logic and web frameworks.	logic inside router). Good configuration and setup instructions.	without troubleshooting.	es or broken startup.
2. Functional Code Delivery (3 Stories)	**25%**	All 3 stories are fully implemented, running, and functionally verified. Business logic accurately solves the module problem. Invariants strictly upheld. Zero placeholder stubs.	All 3 stories implemented; minor edge conditions unhandled (e.g., missing custom error payload). Core flows functional.	2 of 3 stories functional, or 3 stories present but relying on mocked in-memory hardcoding where DB was required.	Incomplete code; endpoints crash on execution; core business rules not implemented.
3. Automated Testing & Verification	**20%**	Comprehensive unit & integration tests mapped directly to BDD Gherkin scenarios. Tests execute fast, deterministically, and assert domain invariants and failure states.	All 3 stories have passing tests covering happy paths. Minor gaps in edge-case or error response testing.	Minimal tests (e.g., only 1 test per story); asserts only trivial HTTP 200 without checking response body semantics.	No tests, broken test suite, or tests that pass vacuously without assertions.
4. BMad Full	**20%**	Exemplary execution of the	Followed build	Superficial ceremony;	Zero evidence of

Assessment Dimension	Weight	Outstanding (90–100%)	Competent (75–89%)	Developing (60–74%)	Beginning (<60%)
Ceremony & Governance		6-step build ceremony. sprint-status.yaml is clean and accurate. AI Decision Log documents ≥ 2 critical corrections of AI code.	sequence; sprint-status.yaml maintained. AI Decision Log records prompts with basic review notes.	sprint-status.yaml updated as an afterthought; AI Decision Log lacks critical analysis of AI outputs.	BMad workflow; raw uncontrolled AI copy-pasting without audit trail.
5. Code Craft, Cleanliness & Docs	**15%**	Highly idiomatic Python/TypeScript. Proper typing (Pydantic/TypeScript interfaces), informative logging, no dead code, crisp README and walkthrough report.	Clean, readable code with minor linting or typing oversights. Clear README with reproducible setup.	Inconsistent naming, missing type hints, sparse documentation, or cluttered repository.	Spaghetti code, unformatted, missing README, zero code comments or explanation.

Annex: The 12 Subsystem Modules & 3 Recommended Core Stories

Each student triad implements the **3 core stories** detailed below for their assigned module. If the recommended stories are not feasible, other similar stories may be implemented.

Module 1: User, Role, and Subject Access Manager

Focus: Identity provisioning, academic subject mapping, and permission checks.

Story 1.1: Single User & Subject Entitlement Provisioning

- **As a** Department Administrator (*Dr. Marcus Vance*),

- **I want to** provision a user account and bind it to an academic subject entitlement (CS-480),
- **So that** the student can authenticate and be recognized as an authorized participant in the course.
- **Technical Context:** Inbound POST /api/v1/users and POST /api/v1/entitlements; Schema: users(id, username, email, role), subject_entitlements(id, user_id, subject_id, role, status).
- **Technical Tasks:**
    1. Define domain entity User with roles (Student, Instructor, Admin).
    2. Implement repository port & SQLite/Postgres adapter with unique constraint on (user_id, subject_id).
    3. Create REST controller with input validation (valid email format, non-empty subject code).
- **BDD Acceptance Scenario:**

```
Scenario: Successfully provision student user with subject entitlement
  Given an administrator with valid credentials
  When a request is submitted to create user "sofia_ml" with role "Student" and subject "CS-480"
  Then the system creates the user record returning HTTP 201
  And an active entitlement for "CS-480" is linked to the user
  And the user ID is returned in the response payload
```

Story 1.2: Subject Access Verification Query

- **As an** API Service Worker,
- **I want to** query whether a specific user holds an active entitlement for a given subject code,
- **So that** downstream compute services can gate model invocation in < 20 ms.
- **Technical Context:** Inbound GET /api/v1/users/{user_id}/entitlements/{subject_id}/verify.
- **Technical Tasks:**
    1. Implement fast domain query service checking active subject enrollment.
    2. Return explicit verification result { user_id, subject_id, is_authorized, role }.
    3. Handle unknown user ID or un-enrolled subject returning { is_authorized: false } with HTTP 200.
- **BDD Acceptance Scenario:**

```
Scenario: Verify active subject enrollment
  Given a user "usr_102" enrolled in "CS-480" with status "ACTIVE"
  When the gateway queries verification for "usr_102" in subject "CS-480"
```

```
    Then the system responds with HTTP 200
    And the response body contains "is_authorized: true" and "role: Student"
```

Story 1.3: Immediate Entitlement Revocation on Course Drop

- **As a** Course Registrar,
- **I want to** revoke a student's entitlement when they withdraw from a course,
- **So that** their access to the course model pool terminates immediately.
- **Technical Context:** Inbound DELETE /api/v1/users/{user_id}/entitlements/{subject_id}; Invariant: Revocation must be instantaneous and idempotent.
- **Technical Tasks:**
    1. Implement revocation use-case in domain service (transitions status to REVOKED or removes record).
    2. Emit revocation audit log event with timestamp and registrar ID.
    3. Ensure subsequent verification queries for that subject return is_authorized: false.
- **BDD Acceptance Scenario:**

```
Scenario: Revoke course access for dropped student
  Given an active entitlement for user "usr_102" in "CS-480"
  When the registrar deletes the entitlement for "usr_102" in "CS-480"
  Then the system returns HTTP 200 with status "REVOKED"
  And subsequent verification queries for "usr_102" in "CS-480" return "is_authorized: false"
```

Module 2: Academic GPU Policy Engine

Focus: Deterministic policy evaluation, time-window gates, and explainable decisions.

Story 2.1: Baseline GPU Access Request Evaluation

- **As an** Inference Gateway or Lab Coordinator,
- **I want to** evaluate a compute request against core user permissions and GPU availability,
- **So that** valid jobs receive an immediate ALLOW decision with cited rationale.
- **Technical Context:** Inbound POST /api/v1/policy/evaluate; Schema: policy_rules(id, name, target_role, allowed_gpu_type, priority).
- **Technical Tasks:**
    1. Implement rule evaluation engine matching request parameters (role, gpu_type, model_tier).

2. Return structured decision object: { decision: "ALLOW" | "DENY", rule_id, rationale }.
3. Ensure evaluation executes deterministically in < 30 ms.

- **BDD Acceptance Scenario:**

```
Scenario: Approved evaluation for standard student workstation job
  Given a policy rule allowing role "Student" to use "24GB_WORKSTATION"
  When an evaluation request arrives for user "mateo" (role "Student", requested
"24GB_WORKSTATION")
  Then the engine returns decision "ALLOW"
  And cites the rule ID and reason "Standard student lab policy satisfied"
```

Story 2.2: Time-Window Lab Class Protection Gate

- **As a** Lab Director (*Dr. Rebecca Reed*),
- **I want to** reject asynchronous student jobs during scheduled laboratory class hours,
- **So that** physical lab students have unencumbered real-time access to machines.
- **Technical Context:** Domain policy rule incorporating simulated wall-clock; Invariant: Time-based rules supersede general entitlements.
- **Technical Tasks:**
  1. Create schedule entity LabScheduleWindow(day_of_week, start_time, end_time, subject_id).
  2. Implement time-interceptor evaluating if current_time falls within an active class window.
  3. Return explainable DENY with exact next available window timestamp.
- **BDD Acceptance Scenario:**

```
Scenario: Deny student job during scheduled class hours
  Given a scheduled lab window for "CS-101" on Monday from 10:00 to 12:00
  And the simulated evaluation clock is "Monday 10:30"
  When student "mateo" submits a background GPU request
  Then the engine returns decision "DENY"
  And the rationale states "GPU access restricted during scheduled lab class CS-101"
```

Story 2.3: VRAM Hardware Tier Restriction Gate

- **As a** System Administrator,
- **I want to** deny student requests for 48 GB server resources when their job fits a 24 GB workstation,
- **So that** high-capacity server GPUs are preserved for large-model research.
- **Technical Context:** Evaluation of model weight footprint vs. GPU hardware tiers; Inbound payload includes model_vram_required_gb.

- **Technical Tasks:**
    1. Add rule asserting that model_vram_required_gb <= 24 cannot request 48GB_SERVER unless role is Researcher/Instructor.
    2. Implement descriptive rejection reporting required VRAM and suggested alternative node type.
- **BDD Acceptance Scenario:**

```
Scenario: Reject 48GB GPU request for small model
  Given user "mateo" has role "Student"
  When mateo requests "48GB_SERVER" for a model requiring only 8 GB VRAM
  Then the engine returns decision "DENY"
  And cites reason "Workload fits standard 24GB workstation; server tier restricted"
```

Module 3: GPU Lab Digital Twin

Focus: Live telemetry ingestion, silent node detection, and thermal cordon automation.

Story 3.1: Heartbeat Ingestion & Fleet Node State Tracking

- **As a** Hardware Lab Technician (*Carlos*),
- **I want to** ingest periodic heartbeat telemetry from compute workstations,
- **So that** the platform maintains the current operational state of all 32 machines.
- **Technical Context:** Inbound POST /api/v1/telemetry/heartbeat; Schema: nodes(id, hostname, gpu_model, vram_total_gb, vram_used_gb, temperature_c, status, last_heartbeat_at).
- **Technical Tasks:**
    1. Define node repository initialized with the 32 campus fleet fixtures.
    2. Ingest telemetry payload (node_id, gpu_temp, vram_used_mb, driver_version).
    3. Update node record timestamp and operational metrics in database.
- **BDD Acceptance Scenario:**

```
Scenario: Successfully ingest valid node heartbeat
  Given node "ws-gpu-04" is registered with status "ONLINE"
  When a telemetry heartbeat arrives with temperature 65°C and VRAM used 12288 MB
  Then the system updates "ws-gpu-04" record
  And updates "last_heartbeat_at" to the current timestamp
  And responds with HTTP 202 Accepted
```

Story 3.2: Silent Node Detection & Offline State Transition

- **As a** Systems Operator,
- **I want to** detect when a node fails to report heartbeats for > 3 consecutive intervals,

- **So that** unresponsive machines are marked OFFLINE and excluded from scheduling.
- **Technical Context:** Domain service / scheduled scan evaluate_fleet_liveness(threshold_seconds).
- **Technical Tasks:**
    1. Implement scan routine checking current_time - last_heartbeat_at > (3 * interval).
    2. Transition lagging nodes from ONLINE → OFFLINE.
    3. Generate audit alert record for newly offline machines.
- **BDD Acceptance Scenario:**

```
Scenario: Detect silent node and transition to OFFLINE
  Given node "ws-gpu-12" has last heartbeat 45 seconds ago (interval is 10s, threshold 30s)
  When the liveness evaluation routine runs
  Then node "ws-gpu-12" status is updated to "OFFLINE"
  And an alert event "NODE_SILENT_TIMEOUT" is emitted
```

Story 3.3: Automated Thermal Anomaly Cordoning

- **As a** Lab Safety Engineer,
- **I want to** automatically cordon any node whose GPU temperature exceeds 88°C,
- **So that** hardware degradation or thermal throttling does not corrupt computations.
- **Technical Context:** Interceptor in telemetry ingestion pipeline; Invariant: Thermal threshold is absolute (88°C).
- **Technical Tasks:**
    1. Inspect temperature_c in incoming heartbeat payload.
    2. If temperature_c >= 88, update status to CORDONED_THERMAL.
    3. Refuse any future job placement or allocation queries for that node.
- **BDD Acceptance Scenario:**

```
Scenario: Cordon node experiencing thermal emergency
  Given node "ws-gpu-19" is currently "ONLINE"
  When a heartbeat is received with GPU temperature 91°C
  Then node "ws-gpu-19" is immediately transitioned to "CORDONED_THERMAL"
  And the node is flagged as unavailable for incoming reservations
```

Module 4: GPU Reservation System

Focus: Atomic time-slot bookings, bulk class capacity holds, and conflict detection.

Story 4.1: Individual Workstation Time-Slot Booking

- **As a** Graduate Student (*Lucas*),
- **I want to** book a single GPU workstation for a 4-hour window tomorrow,
- **So that** I have guaranteed compute time for my experimental benchmarks.
- **Technical Context:** Inbound POST /api/v1/reservations; Schema: reservations(id, user_id, node_id, start_time, end_time, status).
- **Technical Tasks:**
    1. Implement reservation creation with start/end time validation (duration ≤ 4 hours).
    2. Query existing bookings for requested node to ensure zero overlap.
    3. Persist reservation with status CONFIRMED.
- **BDD Acceptance Scenario:**

> Scenario: Book available workstation time-slot
> Given workstation "ws-gpu-07" has no bookings between 14:00 and 18:00 tomorrow
> When user "lucas" submits a reservation for "ws-gpu-07" from 14:00 to 18:00
> Then the system creates reservation in status "CONFIRMED"
> And returns reservation ID with HTTP 201

Story 4.2: Reservation Overlap Conflict Detection

- **As a** Student,
- **I want** the system to reject my reservation if another user has already booked the machine,
- **So that** compute resources are never double-booked.
- **Technical Context:** Conflict detection query checking (start_a < end_b) AND (end_a > start_b).
- **Technical Tasks:**
    1. Execute collision query prior to inserting booking record.
    2. Return HTTP 409 Conflict with structured payload detailing conflicting reservation.
- **BDD Acceptance Scenario:**

> Scenario: Reject conflicting reservation
> Given a confirmed reservation for "ws-gpu-07" from 14:00 to 18:00
> When user "alex" attempts to book "ws-gpu-07" from 16:00 to 20:00 on the same date
> Then the system rejects the booking with HTTP 409 Conflict
> And the error payload explains that the requested time slot overlaps an existing booking

Story 4.3: Bulk Workstation Class Reservation

- **As an** Exam Coordinator (*Professor Zhang*),
- **I want to** atomically reserve 20 workstations for a 2-hour laboratory session,
- **So that** my entire class can run hands-on exercises simultaneously.
- **Technical Context:** Inbound POST /api/v1/reservations/bulk; Invariant: All-or-nothing atomicity.
- **Technical Tasks:**
    1. Accept request for count nodes, start_time, end_time, and subject_code.
    2. Find count distinct online nodes with no conflicting reservations during the window.
    3. Insert all reservation records within a single database transaction.
- **BDD Acceptance Scenario:**

```
Scenario: Atomically book 20 machines for class
  Given at least 20 healthy workstations are free on Friday 14:00-16:00
  When Professor Zhang requests a bulk booking of 20 nodes for "CS-101"
  Then the system atomically creates 20 linked reservations
  And returns HTTP 201 with the list of assigned workstation hostnames
```

Module 5: Model Catalog and Entitlement System

Focus: Model metadata registration, subject entitlement scoping, and deprecation.

Story 5.1: Model Registration & Hardware Footprint Validation

- **As an** MLOps Specialist (*Valeria*),
- **I want to** register an approved model with its parameters and minimum VRAM footprint,
- **So that** the university maintains an audited catalog of allowed model weights.
- **Technical Context:** Inbound POST /api/v1/models; Schema: models(id, name, version, format, vram_required_gb, context_length, status).
- **Technical Tasks:**
    1. Validate model registration payload (valid format: GGUF, AWQ, FP16; VRAM > 0).
    2. Store model record with default status ACTIVE.
    3. Compute whether model is compatible with 24 GB workstations or requires the 48 GB server.
- **BDD Acceptance Scenario:**

```
Scenario: Register approved open-weight model
  When admin registers model "Llama-3-8B-Instruct-Q4" with format "GGUF" and VRAM
6.5 GB
  Then the model is stored in the catalog with status "ACTIVE"
  And compatibility tier is flagged as "WORKSTATION_COMPATIBLE"
  And returns HTTP 201 with generated model ID
```

Story 5.2: Subject-to-Model Entitlement Binding

- **As an** Academic Coordinator,
- **I want to** entitle a specific academic subject to invoke a specific model,
- **So that** only students enrolled in authorized classes can see and query that model.
- **Technical Context:** Inbound POST /api/v1/models/{model_id}/entitlements; Schema: model_subject_entitlements(id, model_id, subject_id).
- **Technical Tasks:**
    1. Create mapping record binding model_id to subject_id.
    2. Enforce uniqueness: preventing duplicate bindings.
    3. Provide lookup query returning all models entitled to a given subject.
- **BDD Acceptance Scenario:**

```
Scenario: Entitle subject to model
  Given model "Qwen-2.5-Coder-7B" exists in catalog
  When the coordinator binds "CS-480" to model "Qwen-2.5-Coder-7B"
  Then an entitlement record is created
  And querying entitled models for "CS-480" includes "Qwen-2.5-Coder-7B"
```

Story 5.3: Model Deprecation Lifecycle Transition

- **As a** Catalog Curator,
- **I want to** mark an outdated model version as DEPRECATED with a pointer to its replacement,
- **So that** student workflows are smoothly guided to updated model weights.
- **Technical Context:** Inbound PATCH /api/v1/models/{model_id}/deprecate.
- **Technical Tasks:**
    1. Update model record status to DEPRECATED and attach replacement_model_id.
    2. Return warning metadata on model fetch queries.
- **BDD Acceptance Scenario:**

```
Scenario: Deprecate older model version
  Given model "Mistral-7B-v0.1" is currently "ACTIVE"
  When admin deprecates "Mistral-7B-v0.1" designating "Mistral-7B-v0.3" as
```

```
    replacement
      Then the status changes to "DEPRECATED"
      And requests for model details include the replacement pointer
```

Module 6: Model Distribution Planner

Focus: Pre-cache planning, disk space eviction calculation, and staging.

Story 6.1: Basic Pre-Cache Placement Plan Generation

- **As a** Network Engineer (*Tariq*),
- **I want to** generate a placement plan to pre-cache a 14 GB model across 10 target nodes,
- **So that** nodes are prepared before lab classes start without manual USB copying.
- **Technical Context:** Inbound POST /api/v1/distribution/plan; Invariant: Target node must have sufficient free disk space.
- **Technical Tasks:**
    1. Accept target model_id (with known disk size) and list of target_node_ids.
    2. Inspect node storage state from fleet fixtures; check free_disk_gb >= model_size_gb.
    3. Generate plan object containing ordered list of download tasks.
- **BDD Acceptance Scenario:**

```
Scenario: Generate valid pre-cache plan
  Given model "Qwen-7B" requires 14 GB disk space
  And 10 selected workstations each have >= 50 GB free disk space
  When the distribution planner generates a plan for "Qwen-7B"
  Then the planner outputs a plan with 10 download actions
  And plan status is "FEASIBLE"
```

Story 6.2: Insufficient Storage LRU Eviction Calculation

- **As an** Infrastructure Planner,
- **I want** the planner to identify the least-recently-used (LRU) model caches to evict when disk space is tight,
- **So that** new required models can fit without human intervention.
- **Technical Context:** Algorithmic calculation of candidate evictions based on cached model timestamps.
- **Technical Tasks:**

1. Given a node with insufficient free space for new model M, inspect cached models on that node.
2. Select LRU cached models whose cumulative size clears sufficient disk space.
3. Include explicit EVICT actions prior to DOWNLOAD in the generated plan.

- **BDD Acceptance Scenario:**

```
Scenario: Plan eviction of LRU cache to make space
  Given node "ws-gpu-08" has 8 GB free space
  And the node contains an unused cached model "Old-Llama-7B" (15 GB, last used 20 days ago)
  When a plan is requested to place "New-Model-7B" (14 GB) on "ws-gpu-08"
  Then the generated plan specifies an "EVICT" action for "Old-Llama-7B"
  Followed by a "DOWNLOAD" action for "New-Model-7B"
```

Story 6.3: Aggregate Network Bandwidth Staging

- **As a** Campus Network Administrator,
- **I want to** stagger concurrent downloads so aggregate campus network usage stays under 1 Gbps,
- **So that** student internet connectivity in other buildings is not throttled.
- **Technical Context:** Scheduling plan generator with concurrency limits (e.g., max 2 simultaneous downloads).
- **Technical Tasks:**
  1. Implement staging logic enforcing max concurrent transfers based on link budget.
  2. Assign staggered start times or sequential stages in the output plan JSON.
- **BDD Acceptance Scenario:**

```
Scenario: Stagger downloads under network bandwidth constraint
  Given a bandwidth limit permitting at most 2 simultaneous model downloads
  When a distribution plan is calculated for 6 nodes
  Then the plan divides downloads into 3 sequential waves of 2 nodes each
```

Module 7: Local LLM Inference Gateway

Focus: OpenAI-compatible routing, bearer token validation, and rate limiting.

Story 7.1: Authenticated OpenAI-Compatible Chat Request Routing

- **As a** Student Developer (*Maya*),
- **I want to** submit a standard /v1/chat/completions request with my API bearer token,

- **So that** my student application interacts with local models using standard OpenAI SDKs.
- **Technical Context:** Inbound POST /v1/chat/completions; Headers: Authorization: Bearer <token>; Schema matches OpenAI Chat Completion DTO.
- **Technical Tasks:**
    1. Implement FastAPI /v1/chat/completions route matching OpenAI request/response JSON schema.
    2. Validate incoming bearer token against mock user/token registry.
    3. Route to mock local inference worker and return structured completion object.
- **BDD Acceptance Scenario:**

```
Scenario: Successful authenticated chat completion
  Given a valid bearer token for student "maya"
  When Maya sends a POST request to "/v1/chat/completions" with prompt "Hello, world!"
  Then the gateway responds with HTTP 200
  And the response body matches OpenAI format with choices, message, and usage statistics
```

Story 7.2: Rate-Limit Throttling (HTTP 429)

- **As a** Cluster Administrator,
- **I want to** rate-limit clients to 30 requests per minute,
- **So that** runaway student loops do not saturate inference workers.
- **Technical Context:** Sliding-window or token-bucket rate-limiter middleware.
- **Technical Tasks:**
    1. Implement in-memory rate-limiter keyed by API token or user ID.
    2. When request count exceeds 30 in 60 seconds, intercept request.
    3. Return HTTP 429 Too Many Requests with Retry-After header.
- **BDD Acceptance Scenario:**

```
Scenario: Rate limit exceeded triggers 429
  Given student "maya" has already made 30 requests in the current minute
  When Maya sends request number 31
  Then the gateway immediately returns HTTP 429
  And includes header "Retry-After"
  And the request is not forwarded to the model worker
```

Story 7.3: Unauthorized Model Access Rejection (HTTP 403)

- **As a** Security Auditor,

- **I want to** reject requests for models that the student's subject is not entitled to use,
- **So that** proprietary or restricted research models remain secure.
- **Technical Context:** Interceptor validating requested model against student's entitled model list.
- **Technical Tasks:**
    1. Inspect model field in incoming chat request body.
    2. Check entitlement mapping; if unauthorized, abort request pipeline.
    3. Return HTTP 403 Forbidden with descriptive explanation.
- **BDD Acceptance Scenario:**

```
Scenario: Reject unauthorized model request
  Given student "maya" is enrolled in "CS-101"
  And model "Secret-Research-70B" is restricted to research faculty
  When Maya requests a completion specifying model "Secret-Research-70B"
  Then the gateway rejects the request with HTTP 403 Forbidden
  And returns message "User not entitled to model Secret-Research-70B"
```

Module 8: Token Usage and Quota Service

Focus: Token event metering, daily budget ledger, and hard denial enforcement.

Story 8.1: Token Consumption Event Ingestion & Ledger Recording

- **As a** Quota Service Worker,
- **I want to** ingest token consumption events containing prompt and completion token counts,
- **So that** student and subject usage ledgers are accurately updated in real time.
- **Technical Context:** Inbound POST /api/v1/tokens/events; Schema: token_ledger(id, user_id, subject_id, prompt_tokens, completion_tokens, total_tokens, timestamp).
- **Technical Tasks:**
    1. Implement token event ingestion endpoint accepting token counts.
    2. Atomically increment daily usage counter in user_quotas(user_id, date, tokens_used, token_limit).
    3. Return HTTP 201 with updated remaining balance.
- **BDD Acceptance Scenario:**

```
Scenario: Ingest token consumption event
  Given user "sam" has used 10,000 of their 100,000 daily token limit
  When a token event arrives for "sam" with 500 prompt and 250 completion tokens
  Then the system records the event in the ledger
```

```
    And Sam's total used tokens updates to 10,750
    And remaining balance is reported as 89,250
```

Story 8.2: Soft-Warning Header at 80% Quota Threshold

- **As a** Student (*Sam*),
- **I want to** receive a warning alert when I have consumed $> 80\%$ of my daily token allocation,
- **So that** I know I am close to the limit and can moderate my testing.
- **Technical Context:** Service evaluation returning warning status flags on query.
- **Technical Tasks:**
    1. Calculate percentage consumed on balance query or event ingestion.
    2. When tokens_used / token_limit >= 0.80, return { warning: true, threshold: "80%" }.
- **BDD Acceptance Scenario:**

```
Scenario: Trigger 80% soft warning threshold
  Given user "sam" has a daily limit of 100,000 tokens
  When Sam's total consumption reaches 82,000 tokens
  Then the quota service returns warning status "QUOTA_WARNING_80_PERCENT"
  And indicates 18,000 tokens remain before hard denial
```

Story 8.3: Hard Denial Enforcement at 100% Quota Ceiling

- **As an** Academic Dean (*Gabriel*),
- **I want to** immediately deny compute requests once a student hits 100% of their daily token ceiling,
- **So that** cluster resources are not monopolized by single individuals.
- **Technical Context:** Inbound query POST /api/v1/tokens/check-allowance; Invariant: Zero tolerance for exceeding 100% quota.
- **Technical Tasks:**
    1. Evaluate (current_used + requested) <= token_limit.
    2. If sum exceeds limit, return { allowed: false, reason: "DAILY_QUOTA_EXCEEDED" }.
- **BDD Acceptance Scenario:**

```
Scenario: Reject request when quota is exhausted
  Given user "sam" has consumed 100,000 of 100,000 daily tokens
  When a service checks allowance for Sam requesting 500 tokens
  Then the quota service returns "allowed: false"
  And specifies reason "Daily token quota exhausted; resets at 00:00 UTC"
```

Module 9: Night Training Scheduler

Focus: Daytime job queueing, window activation, and state machine transitions.

Story 9.1: Daytime Job Submission & Queue Staging

- **As a** Senior Research Student (*Kavita*),
- **I want to** submit a deep learning fine-tuning job during the afternoon,
- **So that** it is validated and queued to execute automatically when the night window opens.
- **Technical Context:** Inbound POST /api/v1/jobs; Schema: training_jobs(id, user_id, docker_image, required_vram_gb, estimated_hours, status, submitted_at).
- **Technical Tasks:**
    1. Accept job submission payload (Docker image, required VRAM, priority).
    2. If current simulated time is outside night window (06:00 to 22:00), assign status QUEUED_PENDING_WINDOW.
    3. Return HTTP 202 with job tracking ID.
- **BDD Acceptance Scenario:**

```
Scenario: Queue daytime job submission
  Given the current simulated time is 14:30 (outside 22:00-06:00 night window)
  When Kavita submits a LoRA training job requiring 24 GB VRAM
  Then the system validates the job spec
  And transitions the job to status "QUEUED_PENDING_WINDOW"
  And returns HTTP 202 Accepted
```

Story 9.2: Night Window Opening & Job Dispatching

- **As a** Scheduler Controller,
- **I want to** activate queued jobs when the clock strikes 22:00,
- **So that** idle nighttime GPU capacity is utilized for model training.
- **Technical Context:** State machine transition triggered by scheduler tick on_window_open().
- **Technical Tasks:**
    1. Detect clock entering 22:00 window.
    2. Query jobs in QUEUED_PENDING_WINDOW ordered by submission time.
    3. Match jobs to available healthy workstations and transition status to RUNNING.
- **BDD Acceptance Scenario:**

```
Scenario: Activate queued jobs at 22:00
  Given 3 jobs are in status "QUEUED_PENDING_WINDOW"
```

```
And 5 workstations are currently idle and healthy
When the scheduler clock advances to 22:00
Then the scheduler dispatches the 3 jobs to distinct workstations
And transitions their status to "RUNNING"
```

Story 9.3: Job Execution Status & Progress Tracking

- **As a** Researcher,
- **I want to** poll the status of my queued or running training job,
- **So that** I can observe its state transitions and assigned compute node.
- **Technical Context:** Inbound GET /api/v1/jobs/{job_id}.
- **Technical Tasks:**
    1. Fetch job record by ID.
    2. Return structured status payload including status, assigned_node_id, elapsed_seconds, and estimated completion.
- **BDD Acceptance Scenario:**

```
Scenario: Query active job status
  Given a job "job_88" running on node "ws-gpu-15"
  When user requests details for "job_88"
  Then the system returns HTTP 200
  And response includes status "RUNNING" and assigned node "ws-gpu-15"
```

Module 10: Distributed Training Job Planner

Focus: Gang scheduling feasibility, node allocation, and homogeneous resource validation.

Story 10.1: 4-Node PyTorchJob Feasibility & Allocation Plan

- **As a** Deep Learning Instructor (*Dr. Chen*),
- **I want to** generate a multi-node allocation plan for a 4-worker distributed PyTorch job,
- **So that** workers are assigned to healthy machines with allocated rendezvous ports.
- **Technical Context:** Inbound POST /api/v1/distributed/plan; Schema: distributed_plans(id, job_name, num_workers, master_node_id, worker_node_ids, rendezvous_port, status).
- **Technical Tasks:**
    1. Inspect fleet fixtures for N healthy, unreserved nodes.
    2. Designate node 0 as MASTER and nodes 1..3 as WORKERS.

1. Allocate deterministic rendezvous port (e.g., 29500) and return valid placement spec.

- **BDD Acceptance Scenario:**

```
Scenario: Generate 4-worker distributed training plan
  Given 4 healthy, unallocated 24 GB workstations exist in the fleet
  When a request is submitted for a 4-worker PyTorchJob named "dist-bert"
  Then the planner returns status "FEASIBLE"
  And assigns 1 master node and 3 worker nodes
  And specifies rendezvous port 29500
```

Story 10.2: Heterogeneous Hardware Infeasibility Rejection

- **As a** Graduate Student (*Leo*),
- **I want** the planner to reject my distributed request if the fleet cannot satisfy uniform VRAM requirements,
- **So that** I do not launch distributed training runs that will crash due to hardware mismatches.
- **Technical Context:** Resource validation rule asserting homogeneous GPU VRAM across all distributed workers.
- **Technical Tasks:**
  1. Given request for K workers with V GB VRAM each, check available inventory.
  2. If total available nodes with $\geq V$ GB is $< K$, return infeasibility proof.
  3. Return HTTP 422 Unprocessable with detailed deficit breakdown.
- **BDD Acceptance Scenario:**

```
Scenario: Reject infeasible 48GB multi-node request
  Given the fleet has only 2 GPUs with 48 GB VRAM (on the single server)
  When a user requests 4 distributed workers each requiring 48 GB VRAM
  Then the planner returns status "INFEASIBLE"
  And provides explanation "Requested 4 x 48GB GPUs, but cluster only contains 2"
```

Story 10.3: Gang Scheduling Atomicity Enforcement

- **As a** Cluster Scheduler,
- **I want to** guarantee all-or-nothing (gang) allocation,
- **So that** a job never holds 3 nodes idle while waiting indefinitely for a 4th node.
- **Technical Context:** Invariant: Gang scheduling requires 100% quorum; partial allocations are strictly prohibited.
- **Technical Tasks:**
  1. Attempt to lock all requested workers simultaneously.

2. If any node becomes unavailable during evaluation, abort entire plan and release any temporary holds.

- **BDD Acceptance Scenario:**

```
Scenario: Abort allocation when quorum cannot be formed
  Given a request for a 4-worker job
  And only 3 healthy nodes are currently available
  When the gang scheduler evaluates the request
  Then the allocation is rejected completely
  And zero nodes are locked or held in reserve
```

Module 11: AI Infrastructure Observability Console

Focus: Real-time telemetry aggregation, metric heatmap models, and threshold alerts.

Story 11.1: Multi-Node Telemetry Ingestion & Real-Time Aggregation

- **As a** Cloud & Systems Engineer (*Liam*),
- **I want to** ingest and aggregate GPU utilization and temperature streams from all 32 nodes,
- **So that** the platform maintains current cluster-wide operational statistics.
- **Technical Context:** Inbound POST /api/v1/metrics/stream; In-memory metrics buffer aggregating min, max, avg.
- **Technical Tasks:**
  1. Ingest metric batch containing (node_id, gpu_util_pct, mem_used_mb, temp_c).
  2. Compute rolling 1-minute cluster-wide averages for GPU utilization and memory.
  3. Expose current aggregated snapshot via GET /api/v1/metrics/cluster-summary.
- **BDD Acceptance Scenario:**

```
Scenario: Ingest and aggregate cluster telemetry
  When telemetry batches are ingested for all 32 nodes
  Then the system updates current cluster metrics
  And the cluster summary endpoint returns average GPU utilization and total memory consumed
```

Story 11.2: Fleet Thermal & Utilization Heatmap Matrix

- **As an** Operations Technician,
- **I want to** query a structured heatmap matrix representing the health of all 32 machines,
- **So that** operators can instantly spot hot spots or underutilized workstations.

- **Technical Context:** Inbound GET /api/v1/metrics/heatmap; Output: array of 32 node status objects.
- **Technical Tasks:**
    1. Map all 32 fleet nodes to their latest recorded metrics.
    2. Categorize temperature into color bands: NORMAL ($< 75°C$), WARM ($75–84°C$), CRITICAL ($\geq 85°C$).
    3. Return structured matrix JSON sorted by workstation index.
- **BDD Acceptance Scenario:**

```
Scenario: Fetch cluster heatmap matrix
  Given telemetry exists for all 32 nodes
  When the operator queries "/api/v1/metrics/heatmap"
  Then the system returns an array of exactly 32 node items
  And each item includes node_id, hostname, temperature_band, and utilization_pct
```

Story 11.3: Token Throughput Spike & Latency Alarm

- **As a** Performance Monitoring Engineer,
- **I want the system to** trigger a visual alert when token throughput spikes abnormally,
- **So that** network and compute saturation can be investigated before services crash.
- **Technical Context:** Threshold evaluation service; Invariant: Spike $> 5 \times$ baseline triggers alarm.
- **Technical Tasks:**
    1. Ingest token rate metrics.
    2. Detect rate exceeding threshold (e.g., $> 10{,}000$ tokens/second).
    3. Create active alert object in GET /api/v1/alerts.
- **BDD Acceptance Scenario:**

```
Scenario: Trigger alert on token throughput spike
  When reported token throughput jumps to 15,000 tokens/sec (threshold 10,000)
  Then the monitoring service creates an alert "ALERT_TOKEN_SPIKE"
  And the alert is visible in the active alerts list with timestamp and severity "HIGH"
```

Module 12: AI Operations Copilot

Focus: Natural language query triage, policy citations, and deterministic explainability.

Story 12.1: Natural Language Access Denial Diagnosis

- **As a** Frontline Lab Helpdesk Specialist (*Nadia*),

- **I want to** ask the copilot why a student was denied model access,
- **So that** I receive a cited, deterministic explanation of the governing policy rule.
- **Technical Context:** Inbound POST /api/v1/copilot/query; Payload: { query: "Why was student alex denied access to deepseek-coder?" }.
- **Technical Tasks:**
    1. Parse entity names (user: alex, model: deepseek-coder) from query or test fixture.
    2. Query user entitlements and model catalog policies.
    3. Synthesize human-readable explanation citing exact rule ID (e.g., RULE-CAT-04).
- **BDD Acceptance Scenario:**

```
Scenario: Diagnose model access denial
  Given user "alex" is enrolled in "CS-101"
  And model "deepseek-coder" requires enrollment in "CS-480"
  When Nadia queries "Why was student alex denied access to deepseek-coder?"
  Then the copilot returns diagnosis explaining missing course entitlement
  And explicitly cites policy rule "RULE-CAT-04"
```

Story 12.2: Night Training Schedule Window Inquiry

- **As a** Student,
- **I want to** ask the copilot why my fine-tuning job cannot run at 15:00,
- **So that** I am informed of the institutional night policy and the next open window.
- **Technical Context:** Copilot query inspecting simulated clock and NightSchedulePolicy.
- **Technical Tasks:**
    1. Evaluate query regarding daytime training.
    2. Inspect night window policy (22:00 to 06:00).
    3. Return explanation stating that daytime interactive teaching takes precedence, informing the user of the 22:00 start time.
- **BDD Acceptance Scenario:**

```
Scenario: Explain night schedule policy
  Given simulated time is 15:00
  When a user queries "Why can't Group 4 run their training job right now?"
  Then the copilot responds that training is restricted to the night window
  And states that the next available window opens at 22:00 tonight
```

Story 12.3: Entity Not Found Hallucination Guardrail

- **As a** Systems Auditor,

- **I want the copilot to** respond with a clean “Entity Not Found” error when asked about non-existent users or machines,
- **So that** the copilot never hallucinates imaginary operational facts.
- **Technical Context:** Guardrail validation checking database existence before answering.
- **Technical Tasks:**
    1. Extract requested entities from query.
    2. Verify entity exists in database/fixtures.
    3. If missing, immediately abort synthesis and return structured NOT_FOUND response.
- **BDD Acceptance Scenario:**

```
Scenario: Refuse to hallucinate regarding non-existent user
  When Nadia queries "Why was student ghost_user_99 denied access?"
  And "ghost_user_99" does not exist in the user database
  Then the copilot responds with status "ENTITY_NOT_FOUND"
  And explicitly states "User ghost_user_99 does not exist in cluster records"
```

End of Task Statement for Impl-1. Prepare your environment, initialize your sprint-status.yaml, and launch the BMad Build Full Ceremony.